

Cahier des charges

Projet MediPlan

Plateforme web de gestion des rendez-vous médicaux

Cadre : Projet intégrateur – Programmation informatique

Session : Printemps 2026

Version : 1.0

Date : 28 Mai 2026

1. Présentation du projet

1.1. Contexte

Le projet MediPlan est une plateforme web de gestion des rendez-vous médicaux destinée aux cliniques, aux médecins, au personnel administratif et aux patients. Il s'inscrit dans un contexte où la prise et le suivi des rendez-vous restent, dans une partie des cliniques de proximité du Québec, organisés à l'aide d'outils hétérogènes : appels téléphoniques, agendas papier, fichiers Excel non partagés ou logiciels propriétaires peu interconnectés.

Cette fragmentation engendre plusieurs catégories de problèmes documentés dans la littérature en gestion des services de santé :

- des conflits d'horaires et des doubles réservations dus au manque de synchronisation entre les canaux de réservation (Cayirli & Veral, 2003 ; Dantas et al., 2018) ;
- des rendez-vous manqués (« no-shows »), dont le taux moyen rapporté dans la littérature internationale se situe entre 10 % et 30 % selon les contextes cliniques (Dantas et al., 2018) ;
- une surcharge administrative liée à la prise d'appels, à la confirmation et au rappel manuel des rendez-vous (OCDE, Panorama de la santé, éditions 2021 et 2023) ;
- une difficulté pour les patients à comparer les disponibilités entre cliniques ou médecins, ce qui peut allonger les délais d'accès aux soins de première ligne.

Plusieurs solutions commerciales existent déjà sur le marché québécois et international (Bonjour-Santé, Clic Santé, Chronos, Doctolib en France). Elles démontrent la valeur d'une centralisation des rendez-vous, tout en présentant des limites souvent citées par les cliniques : modèles d'abonnement coûteux, personnalisation restreinte, dépendance à un éditeur externe, intégration limitée avec les processus internes des petites structures. Un comparatif synthétique est proposé en annexe B.

Les utilisateurs ciblés par la plateforme sont :

- le patient, qui réserve, modifie ou annule un rendez-vous ;
- le médecin, qui consulte et gère son horaire et ses disponibilités ;
- l'administrateur de clinique, qui supervise les utilisateurs, les médecins et les statistiques ;
- le super administrateur, qui configure les cliniques et la plateforme.

Le projet est réalisé dans le cadre du projet intégrateur du programme de programmation informatique. Il sert à la fois de support d'évaluation et de démonstration des compétences acquises sur le développement frontend, le développement backend, la conception d'une base de données relationnelle, la sécurité applicative et la conteneurisation.

1.2. Objectifs

Les objectifs ci-dessous sont les résultats que MediPlan doit atteindre. Chaque objectif est associé à un identifiant unique permettant la traçabilité avec les fonctionnalités (§2.2) et les modalités de validation (§5).

Objectifs métiers

- OM-01 — Permettre à un patient de réserver, modifier ou annuler un rendez-vous en moins de 90 secondes en moyenne, sans avoir à téléphoner.
- OM-02 — Permettre à un médecin de consulter son horaire du jour et de la semaine en une seule vue.
- OM-03 — Réduire la charge administrative en automatisant la prise, la confirmation et l'annulation des rendez-vous.
- OM-04 — Éviter les doubles réservations grâce à un contrôle de cohérence systématique côté serveur et côté base de données.
- OM-05 — Centraliser les informations relatives aux rendez-vous tout en garantissant la séparation logique des données entre cliniques.
- OM-06 — Assurer la sécurité des accès en fonction du rôle de chaque utilisateur (patient, médecin, administrateur, super administrateur).
- OM-07 — Fournir aux administrateurs des statistiques utiles au suivi de l'activité (volumes, taux de no-show, taux d'occupation).

Objectifs pédagogiques

- OP-01 — Démontrer la conception et la réalisation d'une architecture client-serveur complète.
- OP-02 — Appliquer les bonnes pratiques de qualité de code (typage statique, linting, revues, tests automatisés).
- OP-03 — Mettre en œuvre une stratégie de sécurité conforme aux recommandations de l'OWASP.
- OP-04 — Concevoir un schéma de base de données relationnelle normalisé et performant.
- OP-05 — Industrialiser le déploiement local avec Docker Compose.

Critères de performance et de qualité évalués

Le projet sera évalué selon les critères transverses suivants :

- la rapidité d'exécution, mesurée côté client (Lighthouse) et côté serveur (tests de charge légers) ;
- la fiabilité des réservations (intégrité des données, absence de conflits) ;
- la qualité du code (couverture de tests, lisibilité, respect du linter, revues de code) ;
- la sécurité des accès et des données (authentification, RBAC, journalisation) ;
- l'ergonomie de l'interface utilisateur (respect des principes Material Design et WCAG 2.1 AA) ;
- la structure et la performance de la base de données (normalisation, indexation, contraintes) ;
- le respect des bonnes pratiques de développement (Git, conventions, intégration continue) ;
- l'adaptation responsive de l'interface (de 360 px à plus de 1920 px).

1.3. Périmètre du projet

Le périmètre est défini de manière à rester réalisable dans la durée d'un semestre. Les éléments inclus correspondent au cœur du produit ; les éléments exclus le sont soit pour des raisons de

complexité réglementaire (dossier médical, prescriptions), soit par dépendance à des partenaires externes (paiement, intégrations hospitalières), soit par non-pertinence pour un projet semestriel (IA médicale, application mobile native).

Ce qui est inclus dans le projet

- l'authentification (inscription, connexion, déconnexion) et la gestion des rôles (RBAC) ;
- la gestion des utilisateurs (création, modification, désactivation) ;
- la gestion des cliniques (configuration, heures d'ouverture, activation) ;
- la gestion des médecins, des spécialités et de leur affectation aux cliniques ;
- la gestion des disponibilités (plages récurrentes, exceptions, congés) ;
- la réservation, la modification et l'annulation de rendez-vous ;
- la gestion du flux clinique du jour (file d'attente, statuts des rendez-vous) ;
- les notifications internes à la plateforme ;
- les tableaux de bord et statistiques par rôle ;
- l'architecture frontend / backend séparée ;
- la base de données relationnelle PostgreSQL ;
- le déploiement local reproductible via Docker Compose.

Ce qui est exclu du projet

- le dossier médical complet (DSE / DSQ) ;
- la gestion des prescriptions ;
- le paiement en ligne ;
- l'application mobile native (iOS / Android) ;
- la synchronisation avec Google Calendar ou tout autre agenda externe ;
- l'intelligence artificielle médicale (aide au diagnostic, triage automatique) ;
- l'intégration avec des systèmes hospitaliers externes (DSQ, Cristal-Net, etc.).

2. Description fonctionnelle

2.1. Besoins et exigences métiers

Problématique à résoudre

Le besoin principal est de centraliser la gestion des rendez-vous médicaux et le flux clinique quotidien d'une clinique de petite ou moyenne taille, aujourd'hui éclaté entre appels téléphoniques, agendas papier et fichiers Excel. La solution doit supprimer les doubles réservations, réduire le temps administratif consacré aux appels et offrir aux patients un canal de réservation autonome accessible 24 h / 24.

Utilisateurs cibles

Quatre rôles distincts interagissent avec la plateforme. Pour chaque rôle, un persona représentatif est décrit ci-dessous afin de guider les choix d'ergonomie et de priorisation.

Rôle / persona	Profil	Besoins principaux
Patient — Marie, 34 ans	Travailleuse de bureau, à l'aise avec son téléphone, peu disponible en journée pour appeler la clinique.	Réserver rapidement, replanifier sans appeler, recevoir une confirmation visuelle claire.
Médecin — Dr Tremblay, 52 ans	Médecin de famille, peu de temps pour des outils complexes, consulte son agenda entre deux patients.	Voir son horaire du jour d'un coup d'œil, marquer un patient « vu » ou « absent », gérer ses absences.
Administrateur de clinique — Sophie, 41 ans	Coordonne l'accueil et la planification, supervise plusieurs médecins, doit régler les conflits rapidement.	Gérer les utilisateurs et les disponibilités, intervenir sur un rendez-vous, consulter les statistiques.
Super administrateur — Karim, 38 ans (équipe MediPlan)	Gère la configuration multi-cliniques et le support technique.	Créer et configurer les cliniques, attribuer les rôles administrateur, surveiller les journaux.

Scénarios d'utilisation

Les scénarios suivants illustrent les actions attendues. Ils servent de base aux tests d'acceptation utilisateur (§5). Un diagramme de cas d'utilisation UML sera joint au dossier de conception.

SC-01 — Réserver un rendez-vous

Acteur principal : Patient authentifié

Préconditions : Le patient dispose d'un compte actif ; au moins une plage est disponible pour la spécialité demandée.

Flux nominal :

1. Le patient sélectionne une spécialité (ou un médecin).
2. Le système affiche les plages disponibles dans les 30 jours à venir.

3. Le patient choisit une plage et confirme.
4. Le système vérifie qu'aucune autre réservation n'occupe la plage.
5. Le système enregistre le rendez-vous et notifie patient et médecin.

Variante ou cas d'erreur : Si la plage est prise entre l'affichage et la confirmation, le système renvoie un message d'erreur et propose d'autres plages.

Postconditions : Le rendez-vous apparaît dans l'agenda du patient et du médecin, avec le statut « réservé ».

SC-02 — Annuler ou modifier un rendez-vous

Acteur principal : Patient (ou administrateur agissant en son nom)

Préconditions : Un rendez-vous futur existe et n'est pas encore commencé.

Flux nominal :

6. L'acteur ouvre la fiche du rendez-vous.
7. Il choisit « annuler » ou « modifier ».
8. Le système vérifie le délai minimum d'annulation (ex. 24 h).
9. Le système enregistre la modification et notifie les parties concernées.

Variante ou cas d'erreur : Si le délai est dépassé, seul l'administrateur peut forcer l'annulation, avec un motif.

Postconditions : Le statut passe à « annulé » ou « replanifié » ; la plage redevient disponible.

SC-03 — Gérer le flux clinique du jour

Acteur principal : Médecin ou administrateur

Préconditions : Au moins un rendez-vous est planifié pour la journée en cours.

Flux nominal :

10. L'utilisateur ouvre la vue « flux du jour ».
11. Il marque les patients « arrivés », « en consultation », « terminés » ou « absents ».
12. Le système met à jour les statistiques en temps réel.

Variante ou cas d'erreur : En cas de retard, l'administrateur peut décaler tout ou partie des rendez-vous d'un médecin via une action groupée.

Postconditions : Le tableau de bord du jour reflète l'état réel de la file d'attente.

SC-04 — Configurer une clinique

Acteur principal : Super administrateur

Préconditions : Le super administrateur est authentifié et autorisé.

Flux nominal :

13. Il crée la fiche de la clinique (nom, adresse, heures d'ouverture, spécialités).

14. Il crée ou rattache les comptes administrateurs et médecins.

15. Le système applique automatiquement les permissions du rôle.

Variante ou cas d'erreur : Une clinique peut être désactivée sans être supprimée pour préserver l'historique.

Postconditions : La clinique apparaît dans la liste, prête à recevoir des réservations.

SC-05 — Consulter les statistiques

Acteur principal : Administrateur de clinique

Préconditions : L'administrateur est authentifié et rattaché à la clinique.

Flux nominal :

16. Il ouvre le tableau de bord « statistiques ».

17. Il choisit une période et un médecin (ou « tous »).

18. Le système affiche volumes, taux de no-show, taux d'occupation, motifs d'annulation.

Variante ou cas d'erreur : Si la période contient peu de données, un avertissement est affiché plutôt qu'un graphique vide.

Postconditions : Les indicateurs sont consultables et exportables.

2.2. Fonctionnalités principales

Le tableau ci-dessous liste les fonctionnalités principales que MediPlan doit contenir. Chaque fonctionnalité est identifiée par un code unique permettant la traçabilité avec les scénarios (§2.1) et les tests d'acceptation (§5). La colonne « Priorité » utilise la méthode MoSCoW (Must, Should, Could) afin d'expliciter le niveau d'engagement de l'équipe pour la durée d'un semestre.

ID	Fonctionnalité	Description	Priorité
EF-01	Authentification et gestion des comptes	Inscription, connexion, déconnexion, réinitialisation de mot de passe, verrouillage après tentatives multiples.	Must
EF-02	Gestion des cliniques	Création, modification, configuration des heures d'ouverture, activation et désactivation.	Must
EF-03	Gestion des médecins et des spécialités	CRUD médecins, affectation à une clinique, gestion des spécialités, gestion des congés.	Must
EF-04	Gestion des disponibilités	Plages récurrentes et ponctuelles, exceptions, absences et fermetures de clinique.	Must
EF-05	Réservation, modification et annulation de rendez-vous	Recherche par spécialité ou médecin, sélection d'une plage, contrôle d'unicité, règles de délai.	Must
EF-06	Flux clinique du jour	Vue temps réel des rendez-vous, statuts (réservé, arrivé, en consultation, terminé, absent), décalage groupé.	Must
EF-07	Notifications internes	Notifications en temps réel des événements pertinents (nouveau rendez-vous, annulation, modification).	Must

EF-08	Tableaux de bord et statistiques	Indicateurs par rôle, filtrables par période et par médecin.	Should
EF-09	Gestion des rôles et permissions (RBAC)	Application stricte du modèle RBAC sur toutes les routes API et toutes les vues frontend.	Must
EF-10	Export CSV des données	Export des rendez-vous d'une période donnée à des fins de reporting et de sauvegarde.	Could
EF-11	Rappels par courriel	Envoi automatique d'un courriel de rappel 24 h avant le rendez-vous (via SMTP local).	Should

2.3. Interface utilisateur

Ergonomie et design de l'interface

L'interface doit être moderne, intuitive et adaptée à des contextes d'usage variés : accueil de clinique, consultation entre deux patients, réservation depuis un téléphone. Trois exigences ergonomiques structurent la conception.

- Simplicité : moins de trois clics pour effectuer la réservation d'un rendez-vous à partir de l'accueil patient.
- Intuitivité : navigation principale toujours visible et adaptée au rôle de l'utilisateur connecté ; feedbacks systématiques (succès, erreur, chargement).
- Adaptation mobile : design responsive natif (Flex Layout, CSS Grid) pour les écrans à partir de 360 px de large.
- Accessibilité : respect des recommandations WCAG 2.1 niveau AA (contraste 4,5:1 minimum, focus visible, alternatives textuelles, navigation complète au clavier).

Technologies recommandées

- Angular (framework SPA) avec TypeScript ;
- Angular Material (bibliothèque de composants accessibles par défaut, conforme à Material Design 3) ;
- HTML5 et CSS3 ;
- Reactive Forms d'Angular pour la validation côté client.

Wireframes

Les wireframes ci-dessous seront produits et annexés au dossier de conception. Ils constituent la base graphique des écrans clés et serviront de référence au développement frontend.

- écran de connexion et d'inscription ;
- tableau de bord par rôle (patient, médecin, administrateur) ;
- écran de réservation patient (recherche, sélection, confirmation) ;
- agenda du médecin (vues jour, semaine, mois) ;
- espace d'administration (cliniques, utilisateurs, statistiques) ;
- vue du flux clinique du jour.

2.4. Conditions d'utilisation

Environnements d'utilisation

- ordinateur de bureau ou portable, sous Windows, macOS ou Linux ;
- tablette à partir de 768 px de largeur ;
- téléphone mobile à partir de 360 px de largeur, via navigateur web (aucune application native).

Navigateurs supportés

- Google Chrome (deux dernières versions majeures) ;
- Microsoft Edge (deux dernières versions majeures) ;
- Mozilla Firefox (deux dernières versions majeures) ;
- Apple Safari (deux dernières versions majeures).

Limites d'utilisation et contraintes

- connexion Internet ou réseau local requise (pas de mode hors ligne) ;
- gestion sécurisée des sessions et des jetons d'authentification ;
- données de démonstration fictives uniquement ; aucune donnée réelle de patient n'est manipulée pendant le projet ;
- déploiement de référence local via Docker Compose ; aucun hébergement cloud requis pour la validation.

3. Description technique

3.1. Technologies à utiliser

Les technologies sont choisies pour leur adéquation au programme de formation, la maturité de leur écosystème et leur représentativité sur le marché québécois. Chaque choix est justifié ci-dessous.

Langages de programmation

- TypeScript pour le frontend Angular et le backend NestJS — typage statique et cohérence entre les deux couches ;
- SQL (dialecte PostgreSQL) pour les migrations et les requêtes complexes ;
- Dockerfile et YAML pour la configuration d'infrastructure (Docker, Docker Compose).

Frameworks et bibliothèques

- Angular et Angular Material pour le frontend (architecture modulaire, composants accessibles, cohérence Material Design 3) ;
- NestJS pour le backend (inspiré d'Angular, intègre validation par décorateurs, documentation OpenAPI, support des tests) ;
- TypeORM ou Prisma comme ORM (migrations versionnées, requêtes paramétrées) ;
- Jest pour les tests unitaires (frontend et backend) ; Cypress ou Playwright pour les tests end-to-end ;
- Helmet pour la sécurisation des en-têtes HTTP ; class-validator pour la validation des DTO.

Base de données

PostgreSQL est retenu comme système de gestion de base de données. Il s'agit d'un SGBD relationnel open-source reconnu pour sa conformité ACID, sa robustesse, son support avancé des contraintes (clés étrangères, contraintes d'unicité partielles, contraintes d'exclusion) et ses index performants. Ces caractéristiques sont essentielles pour garantir l'intégrité des rendez-vous et empêcher toute double réservation au niveau du stockage. Comparé à MySQL ou à une base NoSQL telle que MongoDB, PostgreSQL est mieux adapté à un modèle relationnel comme celui d'une clinique.

Outils transverses

- Git et GitHub (ou GitLab) pour la gestion de version et les revues de code ;
- ESLint et Prettier pour la qualité et l'homogénéité du code TypeScript ;
- Swagger (intégré à NestJS) pour la documentation interactive de l'API ;
- Docker et Docker Compose pour la conteneurisation et le déploiement reproductible ;
- GitHub Actions pour l'intégration continue (linting, tests, build des images Docker).

3.2. Architecture du système

MediPlan repose sur une architecture client-serveur en trois tiers, dans laquelle le frontend, le backend et la base de données sont des composants distincts qui communiquent par des interfaces clairement définies. Cette séparation favorise la maintenabilité, la testabilité et l'évolutivité du système.

Description textuelle de l'architecture

Le flux nominal de communication est le suivant :

Utilisateur (navigateur) → Frontend Angular (SPA) → API REST NestJS → PostgreSQL

Les trois services sont conteneurisés et orchestrés par Docker Compose. Un volume Docker persistant conserve les données PostgreSQL entre deux redémarrages. En production locale, le frontend est servi par un conteneur Nginx ; en développement, il est servi par le serveur de développement Angular.

Composants principaux

- Frontend Angular : interface utilisateur sous forme de SPA, gestion de l'état avec services Angular et RxJS, consommation de l'API REST via HttpClient, gestion du jeton d'authentification via un intercepteur, validation des formulaires côté client.
- Backend NestJS : exposition d'une API REST documentée OpenAPI, organisation modulaire (auth, users, clinics, doctors, appointments, etc.), couche d'accès aux données via ORM, validation systématique des DTO, gestion centralisée des erreurs, modèle RBAC via Guards et décorateurs personnalisés.
- Base PostgreSQL : schéma relationnel normalisé (3e forme normale minimum), contraintes d'unicité et de clés étrangères, index sur les colonnes de recherche fréquente (clinique, médecin, date), migrations versionnées via l'ORM, séparation logique des données par clinique (colonne clinic_id et filtrage systématique).

Schéma d'intégration

Aucun système externe n'est intégré dans la présente itération (les intégrations type DSQ, Google Calendar, SMS ou paiement sont exclues du périmètre — voir §1.3). Tous les services consommés par la plateforme sont internes au déploiement Docker Compose. Un schéma d'architecture détaillé (composants, flux, ports exposés) sera joint au dossier de conception, accompagné des diagrammes UML (cas d'usage, classes, séquence) et du diagramme entité-association de la base.

3.3. Sécurité

Le contexte applicatif — informations relatives à des rendez-vous médicaux — impose une attention particulière à la sécurité, même dans un cadre académique avec des données fictives. La conception sécurité s'aligne sur les recommandations de l'OWASP (notamment l'OWASP Top 10 et l'OWASP ASVS niveau 1) et s'inspire des principes de la Loi 25 du Québec et de la LPRPDE fédérale.

Authentification et gestion des identifiants

- authentification par identifiant et mot de passe, validée côté serveur ;
- stockage des mots de passe par hachage Argon2id (ou bcrypt avec un coût adapté), jamais en clair ;
- émission d'un JWT à courte durée de vie (access token) et d'un refresh token avec rotation ;
- verrouillage temporaire du compte après un nombre configurable de tentatives échouées ;
- déconnexion côté serveur par invalidation du refresh token.

Gestion des rôles et autorisation

- modèle RBAC à quatre rôles : patient, médecin, administrateur, super administrateur ;
- vérification des permissions par Guards NestJS sur chaque endpoint protégé ;
- séparation stricte des données par clinique (un administrateur ne consulte que sa clinique) ;
- principe du moindre privilège appliqué dès la conception.

Sécurité des données et des communications

- validation et nettoyage systématiques des entrées (class-validator et sanitization) contre XSS et injection ;
- utilisation exclusive de requêtes paramétrées via l'ORM pour empêcher l'injection SQL ;
- protection CSRF par jetons lorsque des cookies de session sont utilisés ;
- en-têtes HTTP de sécurité (CSP, X-Content-Type-Options, X-Frame-Options, Strict-Transport-Security) configurés via Helmet ;
- communications cryptées en production via HTTPS (TLS 1.2 minimum).

Journalisation et audit

- journalisation des actions sensibles (connexion, échec, modification de droits, suppression d'utilisateur, accès aux statistiques) ;
- horodatage et identifiant utilisateur systématiques dans les journaux ;
- conservation des journaux pendant la durée du projet à des fins de démonstration.

Cadre légal de référence

Bien que le projet manipule uniquement des données fictives, la conception s'inspire des principes de la Loi 25 du Québec (entrée en vigueur progressive depuis 2022) et de la LPRPDE fédérale (PIPEDA) : consentement explicite, minimisation des données collectées, droit d'accès et de rectification, journalisation des accès aux données sensibles. Ces principes guident la conception même en l'absence d'obligation directe.

3.4. Performance et scalabilité

Les critères de performance sont définis pour un usage typique d'une clinique de petite à moyenne taille. Ils ne visent pas une exploitation à grande échelle et sont mesurés en environnement de démonstration via les outils standards du navigateur (DevTools, Lighthouse) et via des tests de charge légers côté serveur (Artillery ou k6).

ID	Exigence	Méthode de mesure
ENF-PERF-01	Temps de réponse de l'API REST inférieur à 300 ms au 95e percentile pour les opérations courantes (CRUD rendez-vous, listes paginées).	Tests Artillery ou k6 sur l'environnement local, avec un jeu de données réaliste.
ENF-PERF-02	Chargement initial du frontend inférieur à 2 secondes sur connexion 4G simulée (LCP inférieur à 2,5 s).	Audit Lighthouse en mode « Mobile, Slow 4G ».
ENF-PERF-03	Chargement d'un tableau de bord rempli (100 rendez-vous) inférieur à 1,5 seconde.	Mesure manuelle via DevTools avec un jeu de données fictif.
ENF-PERF-04	Support d'au moins 50 utilisateurs concurrents en démonstration sans dégradation perceptible.	Test de charge ciblé sur les endpoints de réservation.
ENF-PERF-05	Aucune double réservation possible, même en cas d'accès concurrent à la même plage.	Test d'intégration simulant deux réservations simultanées.

Le système est conçu pour pouvoir évoluer horizontalement (plusieurs instances de l'API derrière un répartiteur de charge, base PostgreSQL avec répliques en lecture), mais cette scalabilité reste théorique dans le périmètre académique : elle n'est ni déployée ni testée à grande échelle.

4. Planification et livrables

4.1. Phases du projet

Le projet est découpé en cinq phases successives mais partiellement chevauchantes. Le calendrier ci-dessous s'inscrit dans la durée du semestre Printemps 2026 et pourra être ajusté après la première itération. Chaque phase se termine par un jalon de validation interne avec l'encadrant.

Phase	Période	Contenu principal	Jalon de fin
Phase 1 — Analyse et conception	Semaines 1 à 3	Cahier des charges, diagrammes UML (cas d'usage, classes, séquence), MCD/ERD, wireframes des écrans clés.	Cahier des charges et dossier de conception validés.
Phase 2 — Développement backend	Semaines 3 à 7	Squelette NestJS, authentification, RBAC, modules métiers, schéma PostgreSQL, migrations, validation des DTO, API documentée.	API REST minimale fonctionnelle couvrant les fonctionnalités prioritaires.
Phase 3 — Développement frontend	Semaines 5 à 10	Squelette Angular, écran de connexion, tableau de bord par rôle, écran de réservation, agenda médecin, espace d'administration.	Interfaces principales connectées à l'API.
Phase 4 — Tests, sécurité, qualité	Semaines 9 à 12	Tests unitaires, tests d'intégration, tests E2E, audit Lighthouse, revue sécurité OWASP, correction des bugs, optimisation des performances.	Application stable et conforme aux exigences non fonctionnelles.
Phase 5 — Déploiement et présentation	Semaines 12 à 14	Finalisation du déploiement Docker Compose, rédaction de la documentation technique, préparation de la démonstration finale.	Démonstration finale et remise des livrables.

Pour piloter l'avancement, un tableau de suivi (kanban GitHub Projects) est tenu à jour en continu. Une revue d'avancement hebdomadaire permet de réajuster la priorisation des tâches selon la méthode MoSCoW présentée au §2.2.

4.2. Livrables attendus

L'ensemble des livrables ci-dessous est remis en fin de projet. Le code et la documentation sont déposés sur le dépôt Git, organisé en monorepo avec un sous-dossier pour le frontend et un pour le backend.

ID	Livable
LIV-01	Code source du frontend Angular, avec configuration, tests et fichier README.
LIV-02	Code source du backend NestJS, avec configuration, tests et fichier README.
LIV-03	Schéma PostgreSQL versionné via les migrations de l'ORM.
LIV-04	Documentation technique : architecture, démarrage rapide, conventions, déploiement.
LIV-05	Cahier des charges (présent document) dans sa version finale.

LIV-06	Diagrammes UML (cas d'usage, classes, séquence) et MCD/ERD.
LIV-07	Wireframes des écrans clés.
LIV-08	Application fonctionnelle, démontrable de bout en bout.
LIV-09	Fichier docker-compose.yml et Dockerfiles permettant un démarrage en une seule commande.
LIV-10	Documentation OpenAPI de l'API (générée par Swagger).
LIV-11	Support de présentation finale (diapositives ou équivalent).

5. Modalité de validation

La validation du projet repose sur une stratégie à plusieurs niveaux, inspirée de la pyramide de tests classique. L'objectif est de combiner une base solide de tests automatisés rapides à des vérifications manuelles ciblées et à une mesure des indicateurs définis dans les sections précédentes.

Tests d'acceptation utilisateur

- validation des scénarios SC-01 à SC-05 (§2.1) par des utilisateurs test (camarades ou encadrant) ;
- rédaction de critères d'acceptation au format Gherkin (Given / When / Then) pour chaque fonctionnalité ;
- enquête courte de satisfaction (échelle 1 à 5) auprès de trois à cinq utilisateurs test.

Vérification des fonctionnalités spécifiques

- tests unitaires Jest, côté backend et frontend, sur les services métier, les composants critiques, les Guards et les Pipes. Cible : couverture minimale de 70 % sur les modules métier ;
- tests d'intégration sur l'API NestJS (Jest et Supertest) avec une base PostgreSQL dédiée aux tests, couvrant les scénarios principaux ;
- tests end-to-end (Cypress ou Playwright) sur les parcours utilisateurs critiques : connexion, réservation, annulation, gestion d'agenda ;
- tests de non-régression manuels sur les écrans principaux à chaque jalon.

Tests de performance

- tests de charge légers (Artillery ou k6) sur les endpoints critiques pour valider les exigences ENF-PERF-01 à ENF-PERF-05 (§3.4) ;
- audit Lighthouse sur les écrans principaux (performance, accessibilité, bonnes pratiques) ;
- test de concurrence simulant deux réservations simultanées sur la même plage pour vérifier l'absence de double réservation.

Definition of Done par fonctionnalité

Une fonctionnalité n'est considérée comme terminée que si l'ensemble des critères suivants est satisfait :

- le code a été revu par au moins un autre membre de l'équipe ;
- les tests unitaires et d'intégration associés sont écrits et passent ;
- le linting ne signale aucun avertissement bloquant ;
- la documentation technique est mise à jour si l'API ou la base évoluent ;
- la fonctionnalité est démontrable dans l'application déployée localement ;
- les critères d'acceptation associés à la fonctionnalité sont satisfaits.

Indicateurs de succès

Les indicateurs ci-dessous mesurent la réussite du projet, à la fois sous l'angle métier et sous l'angle pédagogique. Ils sont alignés sur les objectifs énoncés au §1.2.

ID	Indicateur	Cible	Méthode de mesure
KPI-01	Temps moyen pour réserver un rendez-vous depuis l'accueil	Moins de 90 secondes	Mesure manuelle sur 5 utilisateurs.
KPI-02	Taux de doubles réservations sur 1000 réservations simulées	0 %	Test de charge automatisé.
KPI-03	Temps de réponse de l'API au 95e percentile	Moins de 300 ms	Tests Artillery ou k6.
KPI-04	Score Lighthouse Performance	Au moins 90	Audit Lighthouse mobile.
KPI-05	Score Lighthouse Accessibilité	Au moins 90	Audit Lighthouse et vérification WCAG manuelle.
KPI-06	Couverture des tests unitaires backend (modules métier)	Au moins 70 %	Rapport Jest.
KPI-07	Satisfaction des utilisateurs test (échelle 1 à 5)	Au moins 4 sur 5	Enquête courte.

6. Conclusion

MediPlan est une plateforme web moderne dont l'ambition est d'améliorer concrètement l'organisation des rendez-vous médicaux et la gestion du flux clinique quotidien, dans le cadre réaliste d'une clinique de proximité. Le projet répond à un besoin documenté par la littérature en gestion des services de santé et s'inspire des solutions existantes tout en proposant une approche centralisée, auto-hébergée et adaptée à de petites structures.

Au-delà de son intérêt fonctionnel, MediPlan est un support pédagogique complet : il mobilise l'ensemble des compétences attendues d'un projet intégrateur en programmation informatique — conception d'une architecture client-serveur, développement frontend Angular, développement backend NestJS, conception d'une base de données relationnelle, application des bonnes pratiques de sécurité, conteneurisation avec Docker et démarche qualité (tests, revues, intégration continue).

Le présent cahier des charges fixe le cadre du travail à réaliser, les exigences à satisfaire et les modalités de validation. Il sera maintenu vivant tout au long du projet : toute évolution du périmètre ou des exigences entraînera une nouvelle révision documentée.

Hypothèses du projet

Pour rester réalisable dans le cadre académique, le projet repose sur les hypothèses suivantes. Toute remise en cause de ces hypothèses devra entraîner une mise à jour du présent cahier des charges.

- le déploiement de référence est local, via Docker Compose, sur la machine de l'équipe ou un poste de démonstration ; aucun hébergement cloud n'est requis pour la validation ;
- les données utilisées en démonstration sont fictives ; aucune donnée réelle de patient n'est manipulée ;
- le périmètre métier suppose une clinique unique pouvant être dupliquée (multi-cliniques), sans intégration de chaîne d'établissements complexes ;
- les notifications restent internes à l'application ; l'envoi de SMS ou de courriels en production n'est pas un livrable obligatoire ;
- les navigateurs cibles sont les deux dernières versions majeures de Chrome, Edge, Firefox et Safari.

Principaux risques identifiés

Les principaux risques sont identifiés ci-dessous, accompagnés de leur stratégie de mitigation. La matrice sera revue à chaque jalon du projet.

ID	Risque	Probabilité	Impact	Mitigation
R-01	Sous-estimation du temps de développement frontend.	Élevée	Élevé	Découper en EF prioritaires (MoSCoW). Reporter les Could/Should si nécessaire. Revue

				hebdomadaire de l'avancement.
R-02	Difficultés liées à la conception du modèle de disponibilités (récurrences, exceptions).	Moyenne	Élevé	Prototypage rapide du module disponibilités, validation tôt avec un cas réel.
R-03	Indisponibilité ponctuelle d'un membre de l'équipe.	Moyenne	Moyen	Tâches versionnées dans Git, documentation à jour, binômage sur les modules critiques.
R-04	Faibles de sécurité non détectées (injection, fuite de jetons).	Faible	Élevé	Application des checklists OWASP, revue de code croisée, scanner de dépendances (npm audit).
R-05	Problèmes de configuration Docker en démonstration.	Faible	Moyen	Test régulier de docker compose up sur un poste vierge, documentation du démarrage.
R-06	Évolution du périmètre en cours de projet (scope creep).	Moyenne	Moyen	Toute nouvelle exigence passe par une mise à jour formelle du cahier des charges et un arbitrage MoSCoW.

Pistes d'évolution post-projet

Plusieurs pistes d'évolution, exclues de la présente itération, pourraient être envisagées dans un cadre ultérieur :

- intégration de rappels par SMS ;
- synchronisation avec des agendas externes (Google Calendar, Outlook) ;
- mise en conformité poussée à la Loi 25 dans une perspective d'exploitation réelle ;
- ouverture à un réseau de cliniques avec administration multi-niveaux ;
- exposition d'une API publique documentée pour des intégrations partenaires.

Annexes

Les annexes ci-dessous complètent le cahier des charges sans modifier sa structure principale. Elles regroupent les ressources de référence, le comparatif des solutions existantes et le glossaire des termes techniques utilisés.

Annexe A — Sources et références

Les références ci-dessous justifient les éléments de contexte (§1.1), les exigences de sécurité (§3.3) et les choix méthodologiques retenus.

Études et littérature

- Cayirli, T. et Veral, E. (2003). « Outpatient scheduling in health care: A review of literature ». *Production and Operations Management*, 12(4), 519-549.
- Dantas, L. F., Fleck, J. L., Cyrino Oliveira, F. L. et Hamacher, S. (2018). « No-shows in appointment scheduling – a systematic literature review ». *Health Policy*, 122(4), 412-421.
- OCDE. Panorama de la santé, éditions 2021 et 2023 — indicateurs sur l'organisation des soins primaires.
- Commissaire à la santé et au bien-être du Québec. Rapports annuels sur l'accès aux soins de première ligne.

Cadre légal et normatif

- Loi 25 (Québec) — Loi modernisant des dispositions législatives en matière de protection des renseignements personnels (entrée en vigueur progressive depuis 2022).
- LPRPDE / PIPEDA — Loi sur la protection des renseignements personnels et les documents électroniques (Canada).
- OWASP — Top 10 (édition la plus récente) et Application Security Verification Standard (ASVS), niveau 1.
- ISO/IEC 25010:2011 — Qualité du produit logiciel : caractéristiques et sous-caractéristiques.
- W3C — Web Content Accessibility Guidelines (WCAG) 2.1, niveau AA.

Ressources techniques

- Documentation officielle Angular (angular.dev) et Angular Material (material.angular.io).
- Documentation officielle NestJS (docs.nestjs.com).
- Documentation officielle PostgreSQL (postgresql.org/docs).
- Documentation Docker et Docker Compose (docs.docker.com).
- Refactoring UI (A. Wathan, S. Schoger) — principes de conception d'interfaces.

Annexe B — Benchmark des solutions existantes

Le tableau ci-dessous compare quelques solutions existantes de prise de rendez-vous médicaux. Il sert à positionner MediPlan et à justifier ses choix de périmètre. Les informations sont synthétisées à partir des sites éditeurs et de la presse spécialisée.

Solution	Origine	Modèle	Forces	Limites pour une petite clinique
Bonjour-Santé	Québec	Commercial	Adoption au Québec, intégration avec plusieurs cliniques.	Modèle propriétaire, personnalisation limitée.
Clic Santé	Québec — réseau public	Service public (RAMQ)	Gratuit, couverture large dans le réseau public.	Périmètre public, peu adapté à une clinique privée individuelle.
Chronos / Petal Health	Québec	Commercial B2B	Suite complète de gestion clinique.	Coût et complexité élevés pour une petite structure.
Doctolib	France et Europe	Commercial	Très grande maturité produit et UX.	Indisponible au Canada, modèle propriétaire.
MediPlan (présent projet)	Académique	Open / auto-hébergé	Adapté à une petite clinique, code maîtrisé, déploiement local.	Périmètre semestriel limité, pas d'exploitation à grande échelle prévue.

Annexe C — Glossaire

Le glossaire regroupe les termes techniques et métier utilisés dans le cahier des charges. Il s'adresse autant à l'équipe technique qu'aux lecteurs non spécialistes.

Terme	Définition
API REST	Style d'architecture logicielle dans lequel un serveur expose ses ressources via HTTP, en respectant les principes de l'architecture REST.
Argon2 / bcrypt	Algorithmes de hachage de mots de passe résistants aux attaques par force brute, recommandés par l'OWASP.
CRUD	Create, Read, Update, Delete — les quatre opérations fondamentales sur une ressource.
Definition of Done	Ensemble de critères qu'une fonctionnalité doit satisfaire pour être considérée comme terminée.
DTO	Data Transfer Object — objet servant à transporter des données entre les couches d'une application.
JWT	JSON Web Token — jeton signé numériquement, utilisé pour transporter une identité authentifiée entre client et serveur.
MoSCoW	Méthode de priorisation des exigences : Must have, Should have, Could have, Won't have.
No-show	Patient qui ne se présente pas à son rendez-vous et ne l'annule pas au préalable.
ORM	Object-Relational Mapping — couche d'abstraction entre code orienté objet et base de données relationnelle (ex. TypeORM, Prisma).
RBAC	Role-Based Access Control — contrôle d'accès fondé sur les rôles attribués aux utilisateurs.

SLA	Service Level Agreement — engagement chiffré sur la qualité de service (disponibilité, temps de réponse).
SPA	Single Page Application — application web exécutée dans le navigateur sans rechargement complet de la page.
WCAG	Web Content Accessibility Guidelines — référentiel international pour l'accessibilité des contenus web (W3C).